FDD2034
Research Preparation Course

Lecture 4
Modal and Temporal Logic, part 1

Mads Dam

What is Modal Logic About?

View one:

The study and formalization of concepts that go beyond classical (propositional, first-order) logic

such as:

- ϕ is necessary, ϕ is known, ϕ may come to pass, ϕ is provable, …

This is studied in the first instance by coming up with axiomatisations and models that characterize, or not, the concepts we are interested in, as in philosophical logic.

What is Modal Logic About?

View two:

Interesting and relevant properties of computational systems and models

such as:

- Graphs
- Transition systems
- Rewriting systems
- Models of computer systems, SW and HW

In CS we generally study this from the point of view of semantics, i.e. the models.

Kripke Models, Kripke Frames

Kripke models are just graphs with valuations on the states, i.e.

$$M = (W, R, V)$$

where

- W is a set of vertices/worlds/states
- R is a binary relation on W
- X is a set of proposition variables $x \in X$
- $V: X \to \mathcal{P}(W)$ is a valuation of X

We write interchangeably $R(w_1, w_2)$ and $w_1 R\, w_2$

Also often use $\to$ in place of R

A Kripke frame is just a graph

$$F = (W, R)$$

Kripke, Saul A. (March 1959). "A completeness theorem in modal logic". *Journal of Symbolic Logic*. **24** (1): 1–14.

Kripke Semantics

Modal formulas:

$$\phi ::= x \mid \neg\phi \mid \phi \vee \phi \mid \Box\phi$$

Satisfaction:

- $w \models x \ \textit{if} \ w \in V(x)$
- …
- $w \models \Box\phi \ \textit{if for all} \ w', \ \textit{if} \ w \ R \ w' \ \textit{then} \ w' \models \phi$

Note: This is an inductive definition. We're looking for the least solution (under set inclusion), that's why we use just the "if"

Exercise 1 What might happen if we allow other solutions? Give a simple example

Positive Forms

Abbreviate

- $\phi_1 \wedge \phi_2 == \neg(\neg\phi_1) \vee (\neg\phi_2)$
- $\Diamond\phi == \neg(\Box\neg\phi)$

Positive modal formulas:

$$\phi ::= x \mid \neg x \mid \phi \vee \phi \mid \phi \wedge \phi \mid \Box\phi \mid \Diamond\phi$$

Exercise 2

1. Show that
   $w \models \Diamond\phi$ if exists w' such that $w \; R \; w'$ and $w' \models \phi$
1. Show that each modal formula can be rewritten to positive form

Variations and Examples

Multimodal logics:

- Labelled transition systems: $< l > \phi, [l]\phi$
  - $[l]\phi$: For each l-successor, ϕ
- Logics of knowledge: We get back to that

Example 1

Let l range over while programs. Then $\phi_1 \Rightarrow [l]\phi_2$ expresses partial correctness: Whenever started in a state satisfying ϕ_1, if and when l terminates, ϕ_2 will hold

Example 2

Modal logic with labelled modalities and no propositional variables is sometimes called Hennessy-Milner logic. We return to that in a little while.

Validity, Satisfiability, Model Checking

Some main uses of modal (and temporal) logic:

1. Specification of model properties
2. Reasoning, proof, axiomatisations
3. Checking of model properties

For 1. and 3. typically the interpretation of labels and propositional variables/constants is given

For 2:

Validity:

$\models \phi$ if $w \models \phi$ for all models $M = (W, R, V)$ and $w \in W$

Satisfiability:

… (exists a model and world/state) …

Axiomatisation

Definition: Normal modal logic K

The set of modal formulas that

- Contains all propositional tautologies
- Contains all instances of axiom

$$\mathrm{K}\colon \Box(x \Rightarrow y) \Rightarrow \Box x \Rightarrow \Box y$$

- Closed under modus ponens:

$$\mathrm{MP}\colon \frac{\phi \qquad \phi \Rightarrow \psi}{\psi}$$

- Closed under necessitation:

$$\mathrm{Nec}\colon \frac{\phi}{\Box\phi}$$

Soundness and Completeness

Theorem 1 (Soundness of K)
If $\phi \in K$ then ϕ is valid

Exercise 3:
Prove theorem 1

Theorem 2 (Completeness of K)
If ϕ is valid then $\phi \in K$

Exercise 4: (Much harder)
Prove theorem 2.

Proof of Theorem 2, Hints

Instead of theorem 2 prove that if ϕ is consistent then ϕ has a model. What does "consistent" mean?

Model construction idea:

1. States w are maximally consistent sets ∇ of formulas
2. What should R be?
3. We need to obtain a key lemma saying that $\phi \in \nabla$ if and only if $\nabla \models \phi$.
4. How then should we solve 2. ?
5. In order to prove 3. we need a lot of little derivations using the Hilbert-type system K – these are always tricky. Try your hand at some, for instance:

$$\Box(\phi \wedge \psi) \Leftrightarrow \Box\phi \wedge \Box\psi$$

6. Once we've proved 3. are we then done?

Modal Expressiveness

Normal modal logic addresses all Kripke models

Sometimes we're interested in classes of models

Validity wrt class:

Let $\mathcal{C}$ be a class of Kripke models. $\models_{\mathcal{C}} \phi$ if for all models $M = (W, R, V) \in \mathcal{C}$ and $w \in W$, $w \models \phi$

Exercise 5:

Suggest formulas to be added to K in order to characterise the classes of models for which R is:

1. Reflexive
2. Transitive
3. Symmetric

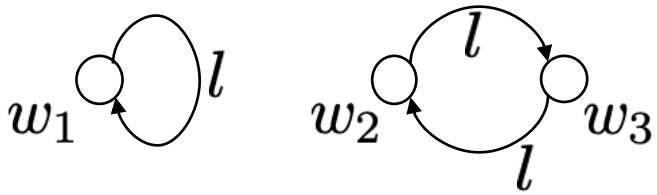
Modal Expressiveness

Not all such “frame conditions” can be expressed

Example 3:

There is no modal formula characterizing the class of irreflexive models

Consider the following two Kripke models



Clearly $w_1 \models \phi$ iff $w_2 \models \phi$ iff $w_3 \models \phi$

Bisimulation

Recall that the relation $B \subseteq W \times W$ is a bisimulation if $w_1 \ B \ w_2$ implies

- for all $x \in X$, $w_1 \in V(x)$ iff $w_2 \in V(x)$
- whenever $w_1 \to w_1'$ then $w_2 \to w_2'$ for some w_2' such that $w_1' \ B \ w_2'$
- whenever $w_2 \to w_2'$ then $w_1 \to w_1'$ for some w_1' such that $w_1' \ B \ w_2'$

Theorem 3 ("Hennessy-Milner")

If there is a bisimulation such that $w_1 \ B \ w_2$ and $w_1 \models \phi$ then $w_2 \models \phi$

There is a partial converse to theorem 3 assuming "image finiteness"

More Bisimulation

Exercise 6: Reprove example 1 using theorem 3.

Bisimulations exist in many variants, strong, weak, branching, etc.

Generally, it is possible to devise logic characterisations similar to theorem 3 by applying modal logic to the appropriately adapted Kripke frames/transition systems.

Logics of Knowledge

In logics of knowledge, aka epistemic logic, the world accessibility relation R is no longer a state transition relation but *equivalence of knowledge/information*.

Equivalence relations are reflexive, symmetric and transitive and characterized by the formulas of exercise 5.

The resulting modal logic is called modal S4

But we need to do more to get at the computational aspects of knowledge

Low and High

One way of agents to learn is to make observations

Partition X into variables L that are observable and variables H that are unobservable.

Define now $w \xrightarrow{l} w'$ by
1. $w \to w'$, and
2. $l = \{x \mid w \in V(x) \text{ and } x \in L\}$

That is, in world w it is possible to observe precisely those x that are in L and that hold in w

J.Y. Halpern, M. Vardi, R. Fagin, Y. Moses: Reasoning About Knowledge. MIT Press 1995

Observation Equivalence

One way of agents to learn is to make observations

Partition X into variables L (low) that are observable and variables H (high) that are unobservable.

Low equivalence:
Let $w \sim_L w'$ if $w \in V(x) \Leftrightarrow w' \in V(x)$ for all $x \in L$

Now revisit:

- $w \models \Box\phi$ if for all w', if $w \sim_L w'$ then $w' \models \phi$
- $w \models \Diamond\phi$ if exists w', $w \sim_L w'$ and $w' \models \phi$

Q: What do these connectives mean?

Epistemic Logic

In epistemic logic usually use $K\phi$ in place of $\Box\phi$

Worlds can have multiple observers/agents, a , with different observation power, i.e. different L's. So:

$$w \models K_a\phi \text{ if for all } w' \text{ if } w \sim_a w' \text{ then } w' \models \phi$$

Exercise 7:

What do these formulas mean?

- $K_aK_bx = 2, K_a(x = y \wedge \neg K_b(x = y))$
- $\neg K_a\phi, \neg K_a\phi \wedge \neg K_a\neg\phi$

Epistemic logic is a rich and interesting topic, but here it comes into its own only once we add dynamics

Temporal Logic?

Formalised properties of time-varying systems

- What time-varying systems?
- What properties?
- Algorithms
- Proof systems

Our tasks:

- Show we can do useful stuff with this
- Understand and compare set-ups for expressiveness and tractability

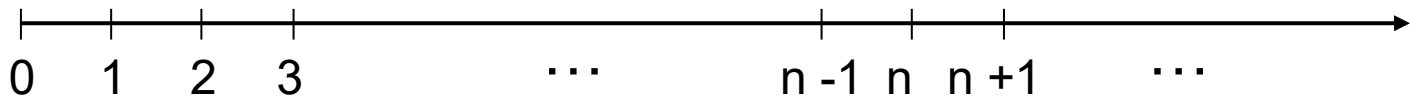
What Time-Varying Systems?

- Continuous real-valued functions?
- Discrete program traces?
- Execution trees?
- Automata?
- Markov chains?
- Java code?
- Distributed processes?
- Real time? Or implicit time?
- Histories or future?
- Finite or infinite?
- Linear or branching? Tree shaped? Graph shaped?

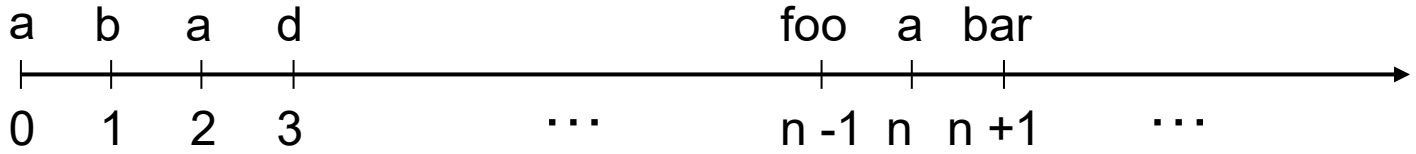
Default Choice – Traces/Paths/Runs

Time is discrete, starts at 0 and goes on forever

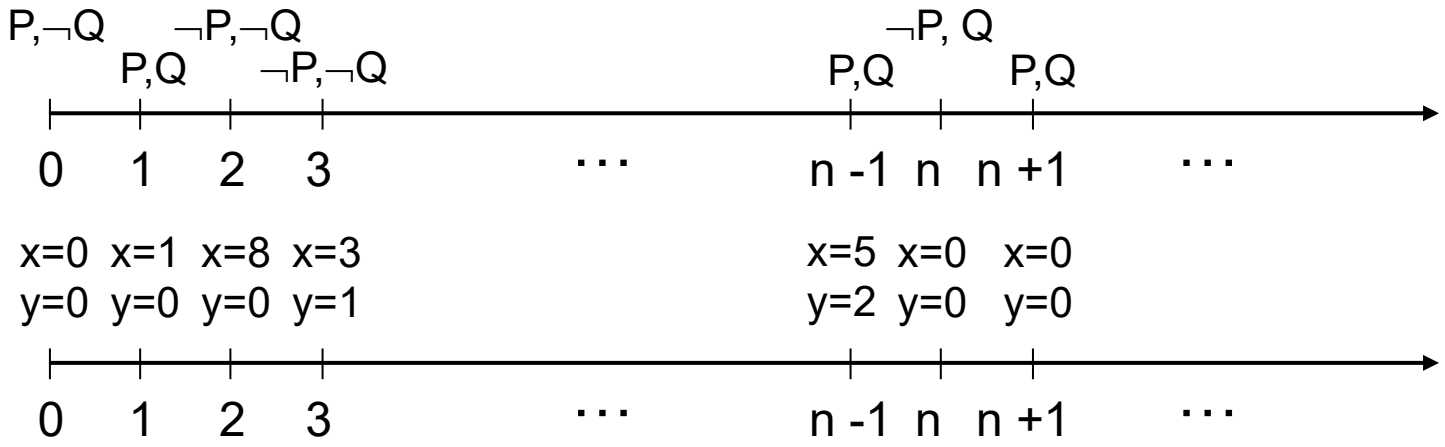
Time points decorated by events



Or conditions/truth assignments/valuations



Or execution traces



How Are Traces Produced?

Maximal runs through a transition system/automaton/frame

- $(S, \rightarrow, S_0)$ transition system
- S set of states
- $\rightarrow \subseteq S \times S$ transition relation, total
- $S_0 \subseteq S$ initial states
- Traces/runs $w = s_0 \rightarrow s_1 \rightarrow \cdots \rightarrow s_n \rightarrow \cdots$

In practice:

- Take your favourite programming/modeling language
- Equip it with discrete transition semantics
- Determine what should be observable events / conditions / execution states
- (Add looping at the end to get traces to be infinite)
- Off you go

Example - Concurrent While Language

Commands:

Cmd ::= skip | x := e | Cmd;Cmd | if e Cmd Cmd
| while e Cmd | await e Cmd | spawn Cmd
| Cmd || Cmd

Stores $\sigma \in x \mapsto_{fin} v \in Val$

Configurations c ::= σ | <Cmd, σ>

Example II

Transitions:

- σ -> σ (… just to get looping …)
- <skip,σ> -> σ
- <x:=e,σ> -> $\sigma[x \mapsto ||e||\sigma]$
- <Cmd_1;Cmd_2,σ> -> <Cmd_1';Cmd_2,σ'>
  if <Cmd_1,σ> -> <Cmd_1',σ'>
- <Cmd_1;Cmd_2,σ> -> <Cmd_2,σ'>
  if <Cmd_1,σ> -> σ'
- (… remaining rules in class …)

Conditions: Boolean/FO expressions in dom(σ_ι)

Traces: c_0 -> c_1 -> c_2 -> … -> c_{n-1} -> c_n -> …

Linear Time Temporal Logic, LTL

Logic of temporal relations between events in a trace:

- Invariably (along this execution) $xy + 3z = k$
- Sometime (along this execution) an acknowledgement packet is sent
- If thread T is infinitely often enabled then T is eventually executed

By no means the last word:

- Last packet received along channel a (along this execution) had the shape (b,c,d) (*past*)
- For all executions (from this state) there is an execution along which a reply is eventually sent (*branching*)
- No matter what choice B made in the past, it would necessarily come to pass that ψ (*historical necessity*)

LTL

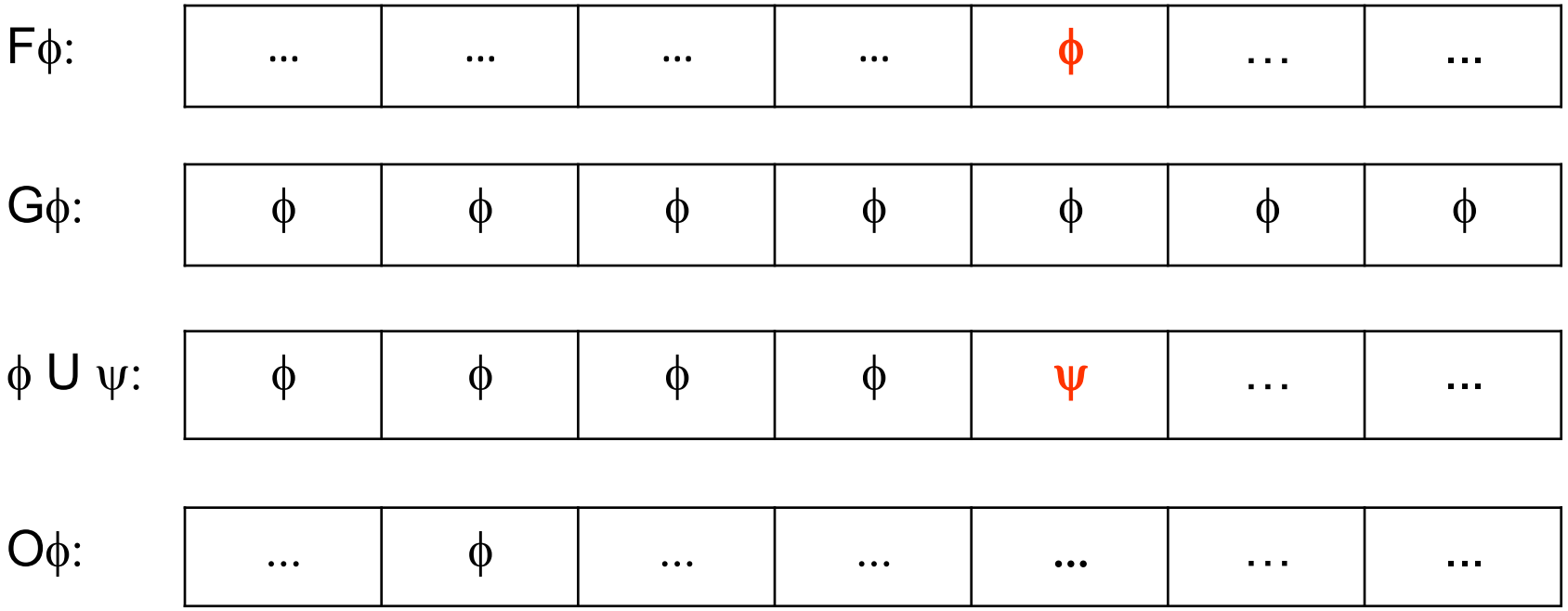
Syntax:

$$\phi ::= x \mid \neg\phi \mid \phi \wedge \phi \mid F\phi \mid \phi\, U \phi \mid O\phi$$

Intuitive semantics:

- x: Propositional variable x holds at the current state
- $F\phi$: At some future time instant ϕ is true
- $G\phi$: For all future time instants ϕ is true
- $\phi\ U\ \psi$: ϕ is true until ψ is true
- $O\phi$: ϕ is true at the next time instant

Pictorially



More on Models

Run w starting in state in S_0
Satisfaction relation $w \models \phi$
$V(x)$: Set of states for which x holds
w^k: k'th suffix of w

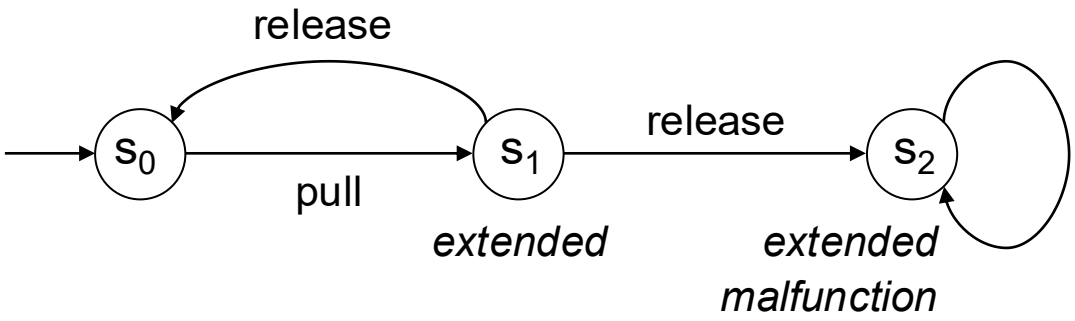
$w \models x$ if $w(0) \in V(x)$
$w \models \neg\phi$ if not $w \models \phi$
$w \models \phi \wedge \psi$ if $w \models \phi$ and $w \models \psi$
$w \models F\phi$ if exists $k \geq 0 . w^k \models \phi$
$w \models G\phi$ if for all $k \geq 0 . w^k \models \phi$
$w \models \phi \; U \; \psi$ if exists $k \geq 0 . w^k \models \psi$ and for all $i : 0 \leq i < k . w^i \models \phi$
$w \models O\phi$ if $w^1 \models \phi$

For transition system $\mathcal{T} = (S, \rightarrow, V, S_0)$, $\mathcal{T} \models \phi$ if $w \models \phi$ for all runs w of $\mathcal{T}$, $w \models \phi$

Examples

- $\phi \vee \psi = \neg(\neg\phi \wedge \neg\psi)$
- $\phi \Rightarrow \psi = \neg\phi \vee \psi$
- $G\phi = \neg F \neg\phi$
- $\phi \; R \; \psi = G\psi \vee (\psi \; U \; (\varphi \wedge \psi))$
  - (sometimes called ”release”)
- $FG\phi$
  - ϕ holds from some point forever = ϕ holds *almost always*
- $GF\varphi$
  - ϕ holds *infinitely often* (i.o.)
- $GF\phi \Rightarrow GF\psi$
  - if φ holds infinitely often then so does ψ
  - Is this the same as $G(F\phi \Rightarrow F\psi)$? As $GF(\phi \Rightarrow \psi)$?
  - As $FG\neg\phi \vee GF(\phi \wedge F\psi)$?

Spring Example

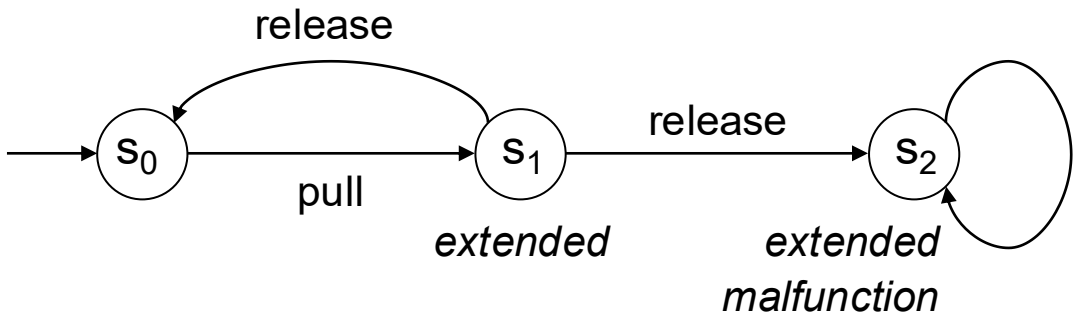


Conditions: *extended, malfunction*

Sample paths:

- $s_0\ s_1\ s_0\ s_1\ s_2\ s_2\ s_2\ \ldots = s_0\ s_1\ s_0\ s_1\ s_2^*$
- $s_0\ s_1\ s_2\ s_2\ s_2\ \ldots = s_0\ s_1\ s_2^*$
- $s_0\ s_1\ s_0\ s_1\ s_0\ s_1\ \ldots = (s_0\ s_1)^*$

Satisfaction by Single Path



$w = s_0 s_1 s_0 s_1 s_2 s_2 s_2 \ldots$

For w:

extended?

O*extended*?

OO*extended*?

F*extended*?

G*extended*?

FG*extended*?

FG*malfunction*?

GF*extended*?

extended U *malfunction*?

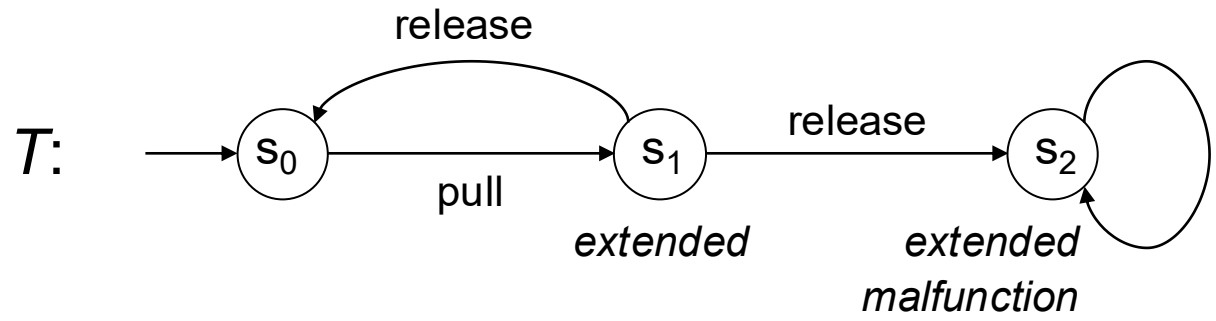
(¬*extended*) U *extended*?

(F*extended*) U *malfunction*?

(F¬*extended*) U *malfunction*?

G(¬*extended* ⇒ O*extended*)

Satisfaction by Transition System



For *T*:

extended?

O*extended*?

OO*extended*?

F*extended*?

G*extended*?

FG*extended*?

FG*malfunction*?

GF*extended*?

extended U *malfunction*?

(¬*extended*) U *extended*?

(F*extended*) U *malfunction*?

(F¬*extended*) U *malfunction*?

G(¬*extended* ⇒ O*extended*)

Exercise 8: Mutex

Assume there are 2 processes, P_l and P_r

State assertions:

- $tryCS_i$: Process i is trying to enter critical section
  E.g. $tryCS_l$: $pc_l = l_4$
- $inCS_i$: Process i is inside its critical section
  E.g. $inCS_l$: $pc_l = l_5 \vee pc_l = l_6$

Express the following properties in LTL:

1. Mutual exclusion, i.e. no two processes are in critical region at the same time
2. Responsiveness, i.e. that whenever a process tries to enter the critical region then it will some time in the future do so
3. That a process keeps trying until access is granted

Example: Fairness

States: Pairs (s, α)

α label of last transition taken, so

$$\frac{s \rightarrow^{\alpha} s'}{(s, \beta) \rightarrow^{\alpha} (s', \alpha)}$$

Σ : Finite set of labels partitioned into subsets

P : "(finite) set of labels of some process" P

State assertions:

- en_P : Some transition labelled $\alpha \in P$ is enabled
- $exec_P$: Label of last executed transition is in P
  i.e. $(s, \alpha) \in V(exec_P)$ iff $\alpha \in P$

Fairness Conditions

Weak transition fairness:

$$\bigwedge_{\alpha \in \Sigma} \neg FG(en_{\{\alpha\}} \wedge \neg exec_{\{\alpha\}})$$

Is this the same as

$$\bigwedge_{\alpha \in \Sigma} FG en_{\{\alpha\}} \Rightarrow GF exec_{\{\alpha\}} \ ?$$

Strong transition fairness:

$$\bigwedge_{\alpha \in \Sigma} GF en_{\{\alpha\}} \Rightarrow GF exec_{\{\alpha\}}$$

Weak process fairness:

$$\bigwedge_{P} \neg FG(en_P \wedge \neg exec_P)$$

Strong process fairness:

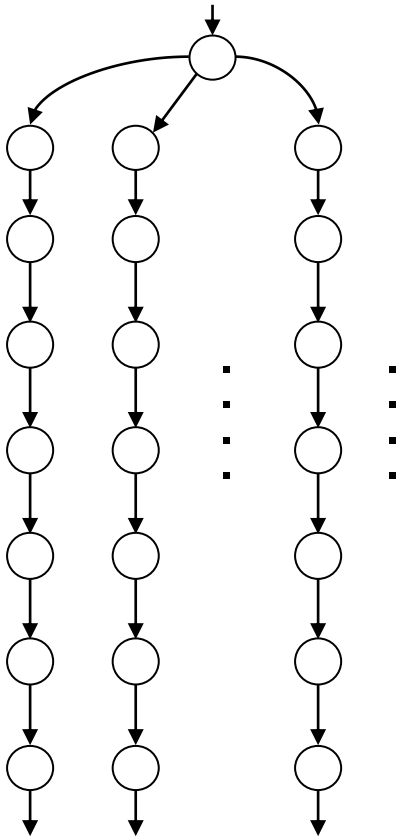
$$\bigwedge_{P} GF en_P \Rightarrow GF exec_P$$

(Many other variants are possible)

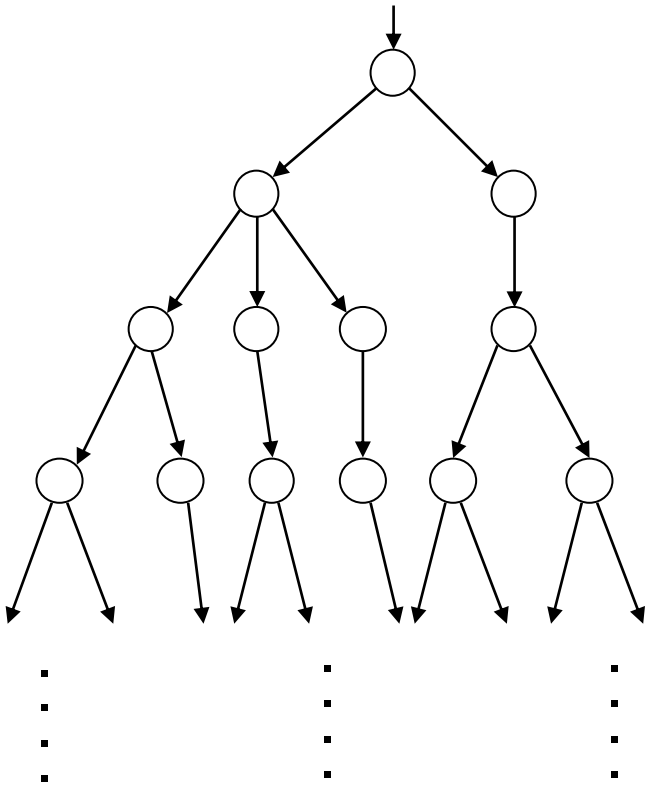
Exercise 9: Which implications hold between these five fairness conditions. Explain.

Branching Time Logic

Sets of paths?



Or computation tree?



Computation Tree Logic - CTL

Syntax:

$$\phi ::= x \mid \neg\phi \mid \phi \wedge \phi \mid AF\phi \mid AG\phi \mid A(\phi U \phi) \mid AO\phi$$

Formulas hold of states, not paths

A: Path quantifier, along all paths from this state

So:

- $AF\phi$: Along all paths, at some future time instant ϕ is true
- $AG\phi$: Along all paths, for all future time instants ϕ is true
- $A(\phi\ U\ \psi)$: Along all paths, ϕ is true until ψ becomes true
- $AO\phi$: ϕ is true for all next states

Note: CTL is closed under negation so also expresses dual modalities EF, EG, EU, EO (E is existential path quantifier).

CTL, Semantics

…

$s \models AF\phi$ if for all w such that $w(0) = s$ exists $k \geq 0$ such that $w(k) \models \phi$

$s \models AG\phi$ if for all w such that $w(0) = s$, for all $k \geq 0$, $w(k) \models \phi$

$s \models A(\phi U \psi)$ if for all w such that $w(0) = s$, exists $k \geq 0$ such that $w(k) \models \psi$ and for all $i : 0 \leq i < k$, $w(i) \models \phi$

$s \models AO\phi$ if for all w such that $w(0) = s, w(1) \models \phi$

Exercise 10: Express the following properties in CTL:

1. No matter what the future might bring it is possible to win a million dollar
2. It is possible to win a million dollar any number of times, but it is also possible to never win anything

CTL – LTL: Brief Comparison

LTL in branching time framework:

- $\phi \mapsto A\phi$ (ϕ to hold for all paths)

CTL $\not\subseteq$ LTL: $EF\phi$ not expressible in LTL

LTL $\not\subseteq$ CTL: $FG\phi$ not expressible in CTL

CTL*: Extension of CTL with free alternation A, F, G, U, O

Advantages and disadvantages:

- LTL often ”more natural”
- Satisfiability: LTL: PSPACE complete, CTL: DEXPTIME complete
- Model checking: LTL: PSPACE complete, CTL: In P

Adding Past

Add to LTL pasttime versions of the LTL future time modalities

Previously, some time in the past, always in the past, since

Theorem 4 (Gabbay's separation theorem): Every formula in LTL + past is equivalent to a boolean combination of "pure pasttime" or "pure future time" formulas

Note: This applies regardless of whether time starts at 0 or at $-\infty$

Theorem 5 (Elimination of past): Pasttime modalities do not add expressive power to LTL

But:

Theorem 6 (Succinctness, LMS'02): LTL + past is exponentially more succinct than LTL

Expressive Completeness

LTL is easily embedded into FOL + linear order

FOL + linear order: First-order logic with 0 and <, unary predicate symbols, and interpreted over ω

Theorem 7 (Kamp'68, GPSS'80, Expressive completeness)

If L is definable in FOL + linear order then L is definable in LTL